

DevOps workflow verification and duration prediction using non-Markovian stochastic Petri nets

Walid Ben Mesmia¹  | Mohamed Escheikh¹ | Kamel Barkaoui²

¹LR-Sys'Com-ENIT, SySCoM-ENIT, ENIT - BP 37, 1002 Tunis Le Belvédère, Tunis, 1002, Tunisia

²Conservatoire national des arts et métiers, CEDRIC-CNAM, 2, rue Conté 75141 Paris Cedex 03, Paris, 75141, France

Correspondence

Walid Ben Mesmia, SySCoM-ENIT, Tunis, Tunisia.
Email: benmesmiawalid77@yahoo.fr

Abstract

In this paper, we provide a non-Markovian Stochastic Petri Net (SPN) model for *DevOps* workflow specification, and we determine how business processes are carried out. After describing our model semantics, we show how general properties related to liveness and safety can be checked. After that, we provide several extensions on SPNs (SPN) the notation and expressivity to check some specific properties related to actors' (Developers and Operators) availability, interactions between the actors, and execution failures detection linked to the *DevOps* steps. Next, we validate the proposed model relevance with MATLAB simulation through a specific *DevOps* case study. Finally, we propose a truncated density function to anticipate the delays related to the *DevOps* business process overall steps.

KEYWORDS

DevOps, NM-WSPN, prediction, specification, verification, workflow

1 | INTRODUCTION

With the *DevOps* emergence¹ as a new software development paradigm, the gap between developers and operational actors should be narrowed by examining organizational and cultural challenges. An integral *DevOps* aspect is to ensure an effective collaboration by establishing highly automated *DevOps* processes.² In addition, the *DevOps* service deployment manuals are very complex, difficult to repeat, and prone to errors.³ The managing component life cycle (install, start, stop, or finish components) of an information technology (IT) infrastructure along the continuous delivery process is nowadays automated using configuration management tools⁴ and infrastructure, such as code.⁵ In industry and research, declarative *DevOps* deployment models are widely used as the most appropriate approach for application deployment and configuration management.⁶

In addition, *DevOps* automation process is, therefore, the key to an effective collaboration and close integration between development and operations. The separation between developers and operations, which is traditionally found in many organizations today, is a major obstacle for fast and frequent application versions. Developers aim to quickly introduce changes to production, while the main goal of the operations staff is to maintain the stability of the production environments.⁷ For this reason, collaboration and communication between developers and operating personnel are mainly based on slow, manual, and error-prone processes.

Thus, there is an orchestration solution variety that requires the corresponding *DevOps* integration tools, such as the *workflows* model or generates scripts describing individual invocations.

However,^{8,9} this is a difficult, costly, and error-prone task as there is no means to do this integration in a standardized manner.

The standard definition lack makes the *DevOps* process implementation (modeling, development, and deployment) difficult and unstable. *DevOps* tools standardization allows for a better collaboration between the actors in development, deployment, and testing and increases efficiency in the delivery process. Thus, the tools presented in Nelson-Smith and Adriansyah and Van Dongen^{10,11} do not specify the collaborative aspect between the actors, the time factor, and the delay in the *DevOps* process progress. To build a *DevOps* process understandable abstraction, we can use an approach specifying it as a business process and modeling it via the Business Process Modeling Notation (BPMN).

The organization's business processes verification and audit are fundamental to its success in a competitive market environment.^{12;13;14} Business process is defined in Delaet et al.,⁴ as a collection of activities that has one or more input types and provides the client with a satisfying output. Equally important, the same definition shows the semantic relationship between the business process inputs (precondition) and the outputs (postcondition). In this definition, the process concept abstraction is obvious. According to Barkaoui and Hicheur and Kheldoun et al.,^{15;16} the BPMN¹⁷ has been widely used as a business process modeling tool. However, it is clear that BPMN 1.0¹⁸ suffers from a standard formal semantic lack. This failure can lead to inconsistencies, ambiguities, and incompleteness within the developed representations.

In BPMN other extensions, substantial enhancements have been provided through the choreography concept. This concept allows the detailed and formalized way description in which the participants personalize their interventions in a business process while coordinating their interactions. Choreography defines interaction between participants through the procedural business contract formalization. In fact, BPMN provides several model types. To describe the different interactions between Dev and Ops in detail, we choose the choreographic model due to its ability to precisely capture such behavior. The choreography process is a new model in BPMN2.0¹⁷ described in the *Choreography Diagram*. It aims to represent interactions between the participants in several formats and focuses on the message flow as an alternative to individual detailed tasks of a process. The Choreography Diagram includes the sender and the receiver within the object (connecting roles instead to a task or using swimlanes) and connects the sent/received messages to this object.

When we speak about the new innovation laws, we can not deny the extent to which standards and technologies have given a boost to the Enterprise Information Systems (EISs),¹⁹ and this resides in its reliable operation. For major firms, the knowledge about business processes is a major means and is stored in EIS. Different firms specify their business processes using different formalisms and tools. The data in the EIS are maintained and used by different actors within a firm, and the business processes correctness should be validated and verified. This generally applies when a process describes a *Workflow*. It is a process including one input point and one output point. If, a process is sound if, for each reachable state (from the initial state), there is a firing sequence leading to a final state.²⁰ In a process model with formal execution semantics, these property types can be precisely defined and automatically verified by tools. Although numerous editors for creating models are used in industry, their analysis facilities often have flaws. Business processes are typically more specified, using other formalisms, than those required by most existing verification tools. BPMN¹⁷ presents a good case for our approach: translating BPMN models to formal standards supported by analysis facilities. In addition, most verification tools work on models based on modeling languages scarcely used in industry. The model transformations²¹ provide more space to apply the model analysis and verification to the business processes used in industry. Then, the original models are transformed into models "as equivalent as possible," specified and verified in other formalisms, allowing the analysis tools to be used for these formalisms. This can give a better representation and overview and offer additional information on the original model quality. Stochastic Petri Nets (SPNs) are well-suited for modeling, analyzing, and describing concurrent systems with a complex process flow. The Petri Net model is capable of mathematically formalizing the DevOps business process. This model in SPN will be used to specify, verify, and validate the properties of liveness, stability ...

The model based on SPNs provides a complete formal analysis tools set that allows for the study of liveness, boundedness, deadlock-freeness, and other properties for control processes.

The purpose of this paper is threefold: On the one hand, modeling a DevOps business process by *Workflow* representation through a high level graphic notation BPMN. The latter will be materialized by a business process specification via a business process diagram. The BPMN graphical notation will be particularly useful to commercial analysts as a flowchart method leading to detailed modeling of the different stages related to the planned DevOps business process. On the other hand, mathematically formalizing the DevOps business process by converting its BPMN representation into an SPN model. This model will be used to verify several properties such as liveness, stability ...

We propose, also, several formal functions to extend SPNs notation and expressiveness in order to verify some specific properties related to the actors (Developer and Operators) availability, interactions between the actors, and execution failures detection related to the DevOps *Workflow* steps.

Third,²² the business process control effectiveness helps to satisfy the objectives of delivering the products and services on time. Therefore, it is necessary to monitor the individual cases progress to be able to detect unwanted gaps in a timely manner and respond accordingly. The process control is essentially based on a case remaining duration prediction and on a deadline missing risk estimate.

Therefore, we come to adopt a standard definition, which is proposed by BPMI,²³ of the *Workflow* domain to cover the business process life cycle (design, deployment, execution, maintenance, and optimization).

So, this paper focuses on the *Workflow* performance aspect through non-Markovian SPNs use. Hence, we propose a non-Markovian SPN extension, named *Non-Markovian Workflow Stochastic Petri Net* (NM-WSPN), favoring the DevOps *Workflow* process life cycle specification, verification, and evaluation.

In other words, our contribution consists of the proposed NM-SPN extension that helps to specify and verify the software-development-business process expressive performance with the DevOps culture. In order to bridge the DevOps gap, we adopt a culture fostering collaboration; besides, we employ learning-based realization methods and a significant method in the automation process. Our NM-WSPN model allows to specify the DevOps *Workflow* process, to control and follow their steps, to detect any execution failure kind, and to set up the cooperative aspects between the actors and their responsibilities in failure case and delay prediction.

The paper's remaining parts are organized as follows. The second section is intended to compare some previous research related to our approach. In Section 3, we define our suggested model basic concepts. Section 4 illustrates the proposed *NM-WSPN* model. Section 5 is devoted to presenting the *DevOps Workflow* process specification, verification, evaluation, simulation, and delay prediction. The last section concludes our work and illustrates the research directions.

2 | RELATED WORKS

In the literature, several research works are related to our article context. Our work covers three main purposes. First, to propose a specification and verification model based on the *NM-SPN* applied in the *DevOps Workflow* context; second, to enrich this model by a delay prediction formal tool. Lastly, we validate the results by a software simulation.

We can classify the related works into two classes:

2.1 | Work related to the *Workflow* specification and verification context

In Barkaoui and Hicheur,¹⁵ the authors propose a model, namely, the recursive *RCATNets* (Recatnets) based on a sound combination of Extended Concurrent Algebraic Term Nets and Recursive Petri Nets, allowing a concise dynamic reconfiguration mechanisms modeling of flexible *Workflow* processes. Also, the RecatNes descriptive power is well-suited for collaborative *Workflows* modeling. Moreover, the use of model checking techniques to prove liveness and safety properties becomes possible due to their semantics defined in conditional rewriting logic. In Kheldoun et al.,¹⁶ the authors find that *BPMN* suffers from a standard formal semantic lack. So, they propose formal *BPMN* semantics using recursive *ECATNets*. In addition, since recursive *ECATNets* semantics are defined in conditional rewriting logic terms, the authors use the Maud *LTL* (Linear Temporal Logic) model checker to check several behavioral properties related to the *BPMN* models. In Yamaguchi et al. and Barkaoui and Ben Ayed,^{24;25} the authors expose the Petri Net structure theory ability to perform a uniform verification for large *subclasses* of parameterized *Workflow* nets modeling control flow patterns associated with control mechanisms, complex synchronization, routing constructs, and resource allocation constraints. They are able to capitalize on these results in areas where collaborative activities and resource competition dominate, such as agile manufacturing systems, Web service applications, and grid computing.

In previous studies,^{26;27;28} they study the synthesis under partial observability for logical specifications on finite marking expressed in *LTLf* (*LTL on finite traces*)/*LDLf* (*LDLf* has exactly the *LTLf* same computational complexity). Actually, they prove that this synthesis form can be considered as a partial observation planning generalization in nondeterministic domains. They show that the usual “belief-state construction” used in partial observability planning is also useful for the general *LTLf*/*LDLf* synthesis. Then, they indicate that the belief state construction can be avoided in a direct automation construction favor that uses projection to hide unobservable propositions.

In Ben Mesmia et al.,²⁹ the authors suggest to extend *NM-SPNs* paradigm to specify the *DevOps Workflow* steps reliability and performance. In order to detect the *DevOps Workflow* steps execution failures, the proposed extension is based on the decomposition of each transition into three microtransitions having specific firing conditions.

Our work is based on the authors' demonstration in Kheldoun et al.¹⁶ they detected the *BPMN*'s suffering from a formal semantic lack. This weakness can lead to inconsistencies, ambiguities, and incompleteness within the developed models.

In this context, we propose to enrich the model presented in Ben Mesmia et al.²⁹ with new formal tools to cover other probabilistic firing transition laws. The verification process adopted by our model is improved by a state space based on the markings cardinality to control and locate the shared resources' conflicts. Our simulation solution must receive, as input, a matrix specifying transitions time at the latest, time at the earliest, and firing rates and parameters related to probability distributions.

2.2 | Delay prediction context-oriented work

The literature analysis related to the prediction context presents two categories:

- Approaches dealing with the process related risks:

Intensifying sensitive managing business processes according to risk in the work context.³⁰ A recent survey on the subject is provided in Suriadi et al.,³¹ which states that the previous work majority has examined the problem from a qualitative point of view. Recently, quantitative approaches have been proposed. The works^{32;33} converge towards the same axis of our approach. In Chauvet,³⁴ the risk indicators for noncompliance with deadlines are defined. This work has the examining and analyzing models objective, such as abnormal activity time, predicting a case of delay. In addition, the authors introduce a risk-based simulation prototype design by simulating the resource threats, such as hardware failures or the business process actor disease.³³

- Remaining time prediction approaches: In the work of Van der Aalst et al.,³⁵ the authors use the information available in the newspapers to predict the business remaining time based on the durations observed in the past. This work serves as a reference point for the prediction method proposed in our work. Also, referring to the works of Van der Aalst et al.³⁵ and Folino et al.,³⁶ they propose an improvement based on predictive clustering. Clustering is done by using additional contextual information. Their approach idea is summed up in similar traces grouping and basic predictions for news with similar features on similar historical *log – journal* cases. Also, they use predictions to warn in a predicted threshold of transgression case.

Other approaches to performance prediction have been done by Hwang et al.³⁷ and Zheng et al.³⁸ They formalize formal tools for calculating QoS criteria, such as expected compounding time, assuming that the service compositions are composed of building blocks.

In the same context, based on the assumption that the activity normal duration having normal distributions, the work of Anklesaria and Drezner³⁹ presents a completion time estimate for PERT network, knowing that PERT nets aim to model projects with purely dependent activities. This modeling elaborates the correlations effects examination between different paths in PERT network to increase the predicted error accuracy taking into account several paths.

Also, the simulation is proposed in an operational decision-making framework in previous studies^{40;41} by building a simulation environment from the current situation and launching a short-term simulation from this state with different simulation parameters.

The analysis of NM-SPN has already been done before. *MonteCarlo* simulation is the preferred choice for analysis, for example, in previous studies.^{41;42} But, the previous works did not cover the durations of a single instance remaining with conditional probability densities.

In this prediction context, to ensure the business processes control, we propose, as in previous studies,^{22;39} a precise remaining case duration prediction and the missing risk late estimation, locating the event between two moments (earlier and later). To do this, we propose to extend our specific model, in Ben Mesmia et al.,²⁹ capable of capturing arbitrary duration distributions, predicting the overall process delay and performing the single step microprediction.

3 | DEFINITIONS AND BASIC CONCEPTS

To introduce our model and the associated formal tools for specifying, verifying, and evaluating the Workflow performance, we define the following concepts.

3.1 | Workflow

According to BPMI,⁴³ a *Workflow* is any activity of choreography involving an interaction between the participants in the form of information exchange. The participants are applications, information system services, human actors, or business processes. To meet the goal of standardization, BPMI proposes a standards set as shown in previous studies^{23;44;45}:

- BPMN defines the graphical notation that allows the designers to specify, in a standard way, the socioeconomic organization processes.
- Business Process Modeling Language (BPML) is a meta-language based on XML (Extensible Markup Language) ensuring the business processes definition in or between the socioeconomic organizations. The BPML language stems from the WSFL,⁴⁶ WSCL,⁴⁷ and WLANG³⁴ efforts.

3.2 | DevOps

*DevOps*⁴⁸ is a neologism representing a movement of *ICT* professionals approaching a different attitude towards software delivery through the collaboration between the software development and the operation functions based on a set of principles and practices such as culture, automation, measurement, and sharing. Another definition⁴⁹ shows that *DevOps* is a development method aimed at bridging the gap between development and operations by focusing on communication and collaboration, continuous integration, quality, and delivery assurance with automated deployment through the use of a set of development practices.

3.3 | Stochastic Petri nets

By introducing a delay in the PNs that aim to design a unique model allowing for both verification and a qualitative and quantitative system analysis, the SPNs have been developed. An SPN is a temporized PN with a probability measure over the trajectories space. The firing sequences are measurable by considering a random space. Formally, an SPN is described in Peterson⁵⁰: An SPN is a couple $S = \langle R, \phi \rangle$, such that

- $R = \langle P, T, Pre, Post, M0 \rangle$ is the underlying PN,
- $\phi : T \rightarrow R^+$, the function that associates a fixed firing rate with each transition,
- $M0$: the PN initial marking.

In order to analyze the complex systems performance, many classes of SPNs are proposed:

- Generalized Stochastic Petri Nets (GSPNs): We distinguish between transitions with a zero delay called immediate transitions and transitions with exponentially distributed random delay called stochastic transitions.^{51;52}
- Deterministic and Stochastic Petri Nets (DSPNs): a Generalized Stochastic Petri Nets extension. It is characterized by immediate transitions, deterministic temporal transitions, and stochastic delay transitions distributed according to exponential laws.⁵³⁻⁵⁶
- Extended Stochastic Petri Nets (ESPNs): A model is formed only by random timed transitions. The time follows a given distribution law. The stochastic process underlying the marking chart is, with some restrictions, a *semi-Markovian* process.⁵⁷

We find that the SPNs do not hold a firing memory. Therefore, we resort to the NM-SPNs.

3.4 | Non-Markovian stochastic Petri nets

Since the 1980s, some extensions have been proved so effective that they are now considered the PNs standard definition part. They are

- The inhibitor arcs: They join a place at a transition and are drawn with a small circle on their destination. A place inhibitor arc p_i has a transition t_k , which deactivates t_k when p_i is not empty.
- The priority transitions: Are integer numbers assigned to transitions. A transition in marking is active only if and only if no higher priority transition is active.
- The Marking-dependent arc Multiplicity: It has been presented in previous studies^{58,59} to model the situations in which the tokens number to be transferred along the arcs (or activating a transition) depends on the system state.

Since the 1980s, many SPN model classes have been developed. They incorporate non-exponential features in their specifications and favor the semantics of the individual memory, which is discussed in Bobbio And Telek.⁶⁰ In order to specify NM-SPN models, which are analytically dealt with, there are three lines of research that can be considered^{61,62}:

- An approach based on the Markov regenerator theory^{15;62},
- An approach based on the additional variables use¹⁵,
- An approach based on the state-space expansion.⁶⁰

The proposed model covers the *workflow* life cycle by taking the SPN approach advantages and the non-Markovian approach to the systems analysis and verification. The following section introduces the proposed model with its adjacent tools for workflow verification and evaluation.

4 | NON-MARKOVIAN WORKFLOW STOCHASTIC PETRI NET MODEL

Our NM-WSPN model is an SPNs extension that introduces new components (places, tokens, and transitions) dedicated to the Workflow context. This section is devoted to the NM-WSPN model basic formal concepts. After defining the model formally, its operating rules are formulated, interpreted, and illustrated. The suggested model adopts the WFMS concepts⁵⁸ and the refinements attributed in Ciardo.⁵⁹ The possible Workflow model activity states belong to the set

{“initiated,” “in-execution,” “active,” “suspended,” “achieved,” “completed,” “archived”}.

4.1 | The model definition

The **Non-Markovian Stochastic Petri Net Workflow**, noted NM-WSPN, is the 13-uplets: $NM-WSPN = \langle P, Tr, A, Ac, SAC, Pre, Post, Disting, Typ-dist, \delta, \varphi, M0, CardM \rangle$

where:

- $P = P_b \cup P_f \cup P_o = \{P_1, P_2, \dots, P_n\}$: finished places set,
- P_b : all initial places,
- P_f : all final places,
- P_o : all ordinary places,
- $Tr = Tr_i \cup Tr_d \cup Tr_s = \{tr_1, tr_2, \dots, tr_n\}$: finished transitions set,
- Tr_i : immediate transitions set.

An immediate transition, having a firing time of zero, specifies an operation whose execution is instantaneous (in zero time)

- Tr_d : deterministic transitions set.

A deterministic transition, having a fixed firing time, specifies a timed operation with a fixed execution duration.

- Tr_s : stochastic transitions set,

A stochastic transition, having a sorting rate that follows a probability law, specifies an operation having a random duration.

- A : *Workflow – actors* non-empty finished set,
- Ac : *Workflow activities* finished set,
- SAC : *subactivities* finished set associated with a *Workflow* activity,
- $Pre: P \times Tr \rightarrow \mathbf{IN}$ incidence application before,
- $Post: P \times Tr \rightarrow \mathbf{IN}$ back incidence application corresponds to the arcs,
- *Disting*: distinguish the function between the transitions' three types: immediate, deterministic, and stochastic,

$$\mathbf{Disting}: T \rightarrow \{1, 2, 3\}; t \mapsto \begin{cases} 1, & tr \in Tr_i \\ 2, & tr \in Tr_d, \\ 3, & tr \in Tr_s \end{cases} \quad (1)$$

- *Typ – dist*: function to distinguish between the three transitions probability distributions (Normal, Log-Normal, and Exponential),

$$\begin{aligned} \mathbf{Typ-dist}: Tr_s &\rightarrow \{e, n, ln\} \\ trs \mapsto \mathbf{Typ-dist}(trs) &= \begin{cases} e, & (\text{Exponential}) \\ n, & (\text{Normal}) \\ ln, & (\text{Log-Normal}) \end{cases}, \end{aligned} \quad (2)$$

- δ : transitions priority function,

$$\delta: Tr \rightarrow \mathbf{IN}; tr \mapsto \begin{cases} 0, & \forall tr \in (Tr_s \cup Tr_d) \\ \delta(tr) \geq 0, & \forall tr \in Tr_i \end{cases}, \quad (3)$$

δ is useful to solve the conflict problems between two immediate transitions.

- φ : function that determines the firing transition rate,

$$\begin{aligned} \varphi: Tr \times \mathbf{IN}^n &\rightarrow \mathbf{IR}^+ \\ \forall tr &\mapsto \varphi(tr) \\ \varphi(tr) &= \begin{cases} 0, & \forall tr \in Tr_i \\ \geq 0, & \forall tr \in Tr_d \\ \text{Density}(tr), & \forall tr \in Tr_s \end{cases}, \end{aligned} \quad (4)$$

- MO : PN s' initial marking is defined as follows:

$$\mathbf{MO}: P \rightarrow A \cup SAC \cup \mathbf{N}^*, \quad (5)$$

- *CardM*: A *CardM* marking cardinality of a NM-WSPN is a family indexed by P , such that $M(p)$ is a $CardM(M(p))$ element. Function that determines the markings number for each marking accessible from M_0 . Formally, *CardM* is written as

$$CardM: P \rightarrow N$$

$$\forall p \mapsto CardM(p)$$

$$CardM(p) = Card(M(p)), \forall p \in P, \quad (6)$$

CardM represents the number of tokens at each Place $\forall p \in P$ after an active $\forall tr \in Tr$ transition firing.

4.2 | Formal tools enriching the model

The model is enriched by functions that build a precondition semantic vision for transition firing to guarantee a qualitative verification during the model transitory dynamics and to ensure the delay predicting functionality. These $\langle Density, Distrib, Com, Execute, MCom, MEx, Truncated \rangle$ formal tools are defined as follows:

- *Density*: A probability density is a function that represents a law of stochastic transition probability (Normal, Log-Normal, or Exponential),

$$Density: Tr_s \times \{e, n, ln\} \rightarrow R^+$$

$$typ = Typ - dist(trs)$$

$$(trs, typ) \mapsto Density(trs) =$$

$$\begin{cases} typ = e, \lambda \cdot e^{-\lambda x} (\lambda \geq 0, x \geq 0); \\ typ = n, 1/(2\sqrt{2\pi}) \cdot e^{-1/2 \cdot x^2}, (x \in R); \\ typ = ln, 1/(x\sigma\sqrt{2\pi}) \cdot e^{-((\ln x - \mu)^2/2\sigma^2)}, \\ (x \in R^+, \sigma \in R^+, \mu \in R); \end{cases} \quad (7)$$

- *Distrib*: function that assigns arbitrary probability distributions to stochastic transitions (Normal, Log-Normal, or Exponential),

$$Distrib: Tr_s \times \{e, n, ln\} \rightarrow R^+$$

$$typ = Typ - dist(trs)$$

$$(trs, typ) \mapsto Distrib(trs) =$$

$$\begin{cases} typ = e, 1 - e^{-\lambda x} (\lambda \geq 0, x \geq 0); \\ typ = n, 1/(2\sqrt{2\pi}) \cdot \int_{-\infty}^t e^{-1/2 \cdot x^2} dx, \\ (x \in R, t \in R); \\ typ = ln, 1/2 + 1/2 \cdot erf^{-1}((\ln x - \mu)^2/2\sigma^2), \\ (x \in R^+, \sigma \in R^+, \mu \in R, \\ erf \text{ is the Gauss error function}); \end{cases} \quad (8)$$

- *Com*: Function determines the value $\langle Wcom, \phi, r \rangle$ that comes from communication between two or more actors involved in the operational subactivity phase in a Workflow process. *Com* is a firing guard for each nonimmediate transition, which will then be transmitted to the tr^c microtransition, according to the decomposition mechanism adopted by the NM-WSPN model.

$$Com: (A \times Ac^2 \times P^2 \times Tr) \rightarrow \{0,1\}^2 \times R^+ \times \{0,1\}, \quad (9)$$

such that,

$$Com(a, A_i, A_j, P_c, P_d, Tr_k) = \langle Wcom, \phi, r \rangle$$

Wcom: Boolean values sequence

r: Boolean value

ϕ : the communication duration between the actors. To formally specify and verify an activity of choreography involving an interaction

TABLE 1 Wex possible interpretations

Wex	Wex signification	Firing possibility
00	No cooperation	Disable
01 / 10	One of the two actors requests to cooperate	Disable
11	The two actors cooperate on the Workflow subactivity execution	Able

TABLE 2 Wcom possible interpretations

Wcom	Wcom signification	Firing possibility
00	No communication	Disable
01/10	One of the two actors requests to communicate	Disable
11	The two actors communicate about the Workflow sub activity	Able

between the actors in an information exchange form, *Com* is a transition *precondition* firing by analyzing the $\langle Wcom, \phi, r \rangle$ values. The Table 1 illustrates the *Wcom* possible interpretations and their influence on the microtransition firing.

- *Execute*: function specifying Workflow *subactivity* execution. *Execute* is a firing guard for each nonimmediate transition, which will then be transmitted to the tr^w microtransition, according to the decomposition mechanism adopted by the NM-WSPN model.

Execute:

$$(\mathbf{SAC} \times \mathbf{Ac} \times \mathbf{P}^2 \times \mathbf{Tr}) \rightarrow \{0,1\} \times \mathbf{R}^+ \times \{0,1\}, \quad (10)$$

such that,

$$Execute(a, A_i, P_c, P_d, Tr_k) = \langle Wex, \phi, a \rangle$$

Wex: Boolean values sequence

a: a Workflow *subactivity*

ϕ : the Workflow *subactivity* execution time to specify and formally verify an activity of choreography involving cooperation between the actors in a Workflow cooperative execution. The Table 2 illustrates the *Wex* possible interpretations and their influence on the microtransition firing.

- *MCom*: communication trace application. *MCom*:

$$(\mathbf{A} \times \mathbf{Tr}) \rightarrow \{0,1\}, \quad (11)$$

such that $MCom(A_i, tr_k) = 1$, if $Com(A_i, , , tr_k) = \langle 11, \phi > 0, 1 \rangle$, 0 if not

ϕ : the Workflow *subactivity* execution time.

This application makes it possible to determine the communication traces matrix.

- *MEx*: execution trace application. *MEx*:

$$(\mathbf{A} \times \mathbf{Tr}) \rightarrow \{0,1\}, \quad (12)$$

such that

$$MEx(A_i, tr_k) = 1, \text{ if } Execute(A_i, , , Tr_k) = \langle 11, \phi > 0, 1 \rangle, 0 \text{ if not}$$

ϕ : the Workflow *subactivity* execution time.

This application makes it possible to determine the execution traces matrix.

- *Truncated*: We define the truncated distribution function as

$$\begin{aligned} &truncated : Tr_s \rightarrow \mathbf{R}^+ \\ &(t_0, t) \in TD \times TD \\ &typ \in \{e, n, ln\} \\ &(tr_s, typ) \mapsto Density(tr_s) \\ &(tr_s, typ) \mapsto Distrib(tr_s) \\ &Truncated(tr_s) = \end{aligned}$$

$$\begin{cases} 0, t \leq t_0, \text{Distrib}(t_0) \leq 1; \\ \text{Density}(t) / (1 - \text{Distrib}(t_0)), \\ t > t_0, \text{Distrib}(t_0) \leq 1; \\ \text{Density}_{\Delta}(t - t_0), \text{Distrib}(t_0) = 1; \end{cases} \quad (13)$$

4.3 | Semantics firing in the NM-WSPN model

Our model noted *NM-WSPN* specifies the activity *DevOps Workflow* behavior via the description of an *SPN* of its life cycle both under normal and abnormal conditions. These are specified by both global failures (at activity level) and local failures (at subactivity level). A subactivity designates a stage among all the stages making up an activity. To capture the different failures, we propose to split each transition (representing a subactivity) into a sequence of three microtransitions. The decomposition/split (transition division) objective is to distinguish between local failures within the same transition. The latter is fired in three successive stages, each representing a microtransition. The microtransitions correspond, respectively, to the tokens consumption from one or more places upstream; the effective transition execution and the new tokens production immediately follow the transition firing.

The firing transitions semantics in the *NM-WSPN* model adopts the same mechanics as that in Ben Mesmia et al.²⁹ So, we indicate with Itr : all transition input places $tr \setminus {}^Itr \subset P$. tr^O : all transition output places $tr \setminus tr^O \subset P$. The model requires that a transition must be divided into three phases, whatever their type is, like in Figures 1 and 5B, at the firing time.

Figures 1 and 5B show the transition splitting approach adopted in our *NM-WSPN* model. Each nonimmediate transition is split into three phases executed successively. The first phase describes the token consumption represented by both the input place(s) set Itr and a microtransition: tr^c (with a firing precondition). The second phase describes the transition execution and is specified by an intermediate place and a microtransition (tr^w). The tr^w firing may be subject to predefined conditions. The cycle ends with the tokens production phase in the tr^O set describing the output place(s) for the activated transition.

When a nonimmediate transition is split into three microtransitions (tr^c , tr^w , and tr^p), there is no risk losing the tokens because the set Itr (respectively, tr^O) of the split transitions is the same as its consumption (respectively, production) microtransition. Also, the microtransitions firing mechanism is sequential, separated by two places (denoted intermediate place), to guarantee the token conservation. Our *NM-WSPN* model enhances the *SPN* expressiveness to allow verifying the firing precondition property of the microtransitions. This allows indicating the token's position in our model (see Figures 1 and 5B). It should be noted that each microtransition inherits (in the programming point of view) the split transition global properties.

In the next section, we will show the techniques application adopted by our model to specify and verify a software development *Workflow* according to the *DevOps* paradigm.

4.4 | NM-WSPN model formal checking

The formal verification that we adopt in our *NM-WSPN* model concerns the detection and success of the system failures based on the tokens game (consumption and production) following the transitions firing via the cardinality marking graph. Generally, the general properties that we check in *PNs* are boundedness, liveness, deadlock (blocking state), reachability, and so forth. To check an *NM-WSPN* model, we adopt an analogy with the verification classical *SPN* concepts. We define the basic formal concepts related to the cardinality markings graph properties:

- *NM-WSPN* model is *pure* if $(\forall p \in P), (\forall tr \in Tr) \setminus Pre(p, tr) \times Post(p, tr) = \emptyset$. This checking is carried for each transition.
- *NM-WSPN* model source transition is verified when the transition $(\exists tr \in Tr)$ is called the (*NM-WSPN*, $CardM_0$) source transition if $cardM^{-1}(tr > \cdot) = \emptyset$. In other words, there is a transition tr that verifies the condition $(\exists tr \in Tr \setminus {}^Itr = \emptyset)$.

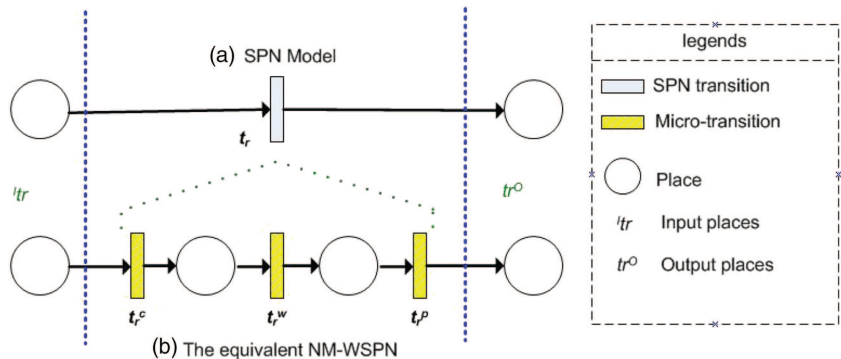


FIGURE 1 Model representation through Stochastic Petri Net (SPN) and Non-Markovian Workflow Stochastic Petri Net (NM-WSPN)

- *NM-WSPN* model well transition is defined, $(\exists tr \in Tr)$ of $(NM-WSPN, CardM_0)$ is a transition verifying the following property if $CardM(tr \succ \cdot) = \emptyset$. In other words, there is a transition tr that verifies the condition $(\exists tr \in Tr \setminus tr^O = \emptyset)$.
- *Quasi-live* transition in *NM-WSPN* model is defined, let $(\forall tr \in Tr)$ be called the *quasi-live* transition of the *NM-WSPN* model, if there is a *CardM* accessible cardinality named $CardM \in NM - WSPN (CardM_0)$ such as $cardM(tr \succ \cdot)$, the couple $(NM - WSPN, CardM_0)$ is said to be almost alive if all transitions are *near-live*.
- $(NM-WSPN, CardM_0)$ is said to be *quasi-live* if and only if for every transition tr of the *NM-WSPN* model, there is, in the markings cardinalities graph, a tr label arc.
- *NM-WSPN* without blocking is defined if it is based on *PNs* and it has markings cardinality. By analogy with the *PNs*, this model is said without blocking if from any Cardinality accessible Marks; there is at least one transition that can be fired such as $\forall CardM \in (NM-WSPN, CardM_0)$, $(\exists tr \in Tr) \setminus CardM(tr \succ \cdot)$. We must verify that the *NM-WSPN* model evolution regardless, we will never block.
- Accessibility problem in a *NM-WSPN* is detected when the model is with markings cardinality $(CardM, CardM')$. The decision that $CardM' \in (NM-WSPN, CardM)$ is the accessibility problem in the *NM-WSPN* model.
- An *NM-WSPN* model is said to be sound if and only if there exists a firing transitions sequence u such that the model remains bounded. Practically, we check the following property:
 $\Delta u = \Delta v \leq 0$
- *NM-WSPN* is bounded, if it has a markings cardinality verifying this condition:
 $(\exists k \in \mathbb{N})$ such that $\forall CardM \in (NM - WSPN, CardM_0)$ and $(\forall p \in P)$, $CardM(p) \leq k$.
- It is verified that an $(NM-WSPN, CardM_0)$ has an infinite sequence if only if the markings cardinality graph has a circuit.
- $(NM-WSPN, CardM_0)$ admits a blocking if the markings cardinality graph contains a vertex without successor.

The main properties that generally studied the *NM-WSPN* model have been stated. Verification consists in determining the techniques for deciding on each of them. Checking based on the model evolution, verified by *NM-WSPN* model, is then presented.

4.5 | Delay prediction by NM-WSPN model

In order to include the delay semantics in the activities description, we define *Event log* associated to *NM-WSPN* model. Let *TD* be the time domain associated to *Tr* transitions set.

We define an *Event Log* by $LEv = (E, , D)$:

- *E*: events finite set,
- *C*: cases finite set where each case specifies a business process instance,
- *D*: $E \rightarrow TD$. A function to assign each event to time-space and classify it as a time function.

The set *E* covers all *NM-WSPN* model transitions *Tr*. The set *C* is included in all *CardM* markings cardinalities graph. To achieve the duration prediction objective, the *NM-WSPN* model is endowed with a duration capture mechanism based on a probabilistic aspect. The *NM-WSPN* model combined with the (*Event log*: *LEv*) is able to predict the remaining durations as well as the transgressing execution deadlines risks.

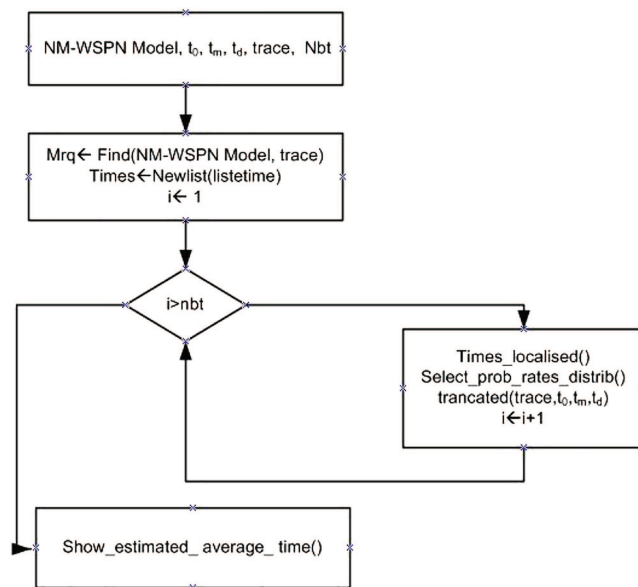
To have a reliable time quantity interpretation by our model, we will introduce the following concepts:

- *Elapsed time*: Let the delay sequence $\{d_0, d_1, d_2, \dots, d_n\}$ such that $d \in TD$ be the time elapsed before the occurrence of e_0 and d_i be the time elapsed between the occurrences of e_i and e_{i+1} ($\forall e_i \in E$). This is the average time to reach a particular state;
- *Sojourn time*: The sojourn in some state must have a value of duration represented by a time interval. This is the average time spent in a particular state;
- *Remaining time*: the average time to reach the end from this state.

The prediction process is based on

- the transitions enrichment with probabilistic delays to deduce the activity duration and the corresponding waiting time,
- weights assignment to transitions to determine the decision outcome probability,
- the probability density function associations with stochastic transitions specifying the firing time distribution,
- using an algorithm that estimates the remaining time in the *NM-WSPN* model based on the elapsed time.

To adapt our prediction approach to the methods that are based on elapsed time, we must apply the following steps:

FIGURE 2 Prediction algorithm organizational chart

- Step 1: remaining time estimation

This step adopts an estimation method like in previous studies.^{63;36} We consider $t_m \in TD$ the moment when the *DevOps* business process could reach the $CardM_F$ final marking cardinality. It is assumed that t_0 is the estimation current time and t_a the current activity a completion time. It is clear that it is dealing with random aspects. The random effect is captured by a variable rated X_M . We propose to estimate the remaining time by determining the relative duration expected, which remains $E(X_M > t_0)$. Actually, one can make the comparison between two estimates $E(X_M > t_0 | t_a > t_0)$ and $E(X_M > t_0)$. In the simultaneous case or synchronous execution between several activities and if t_b represents the second activity duration in execution state at time t_0 , we estimate by $E(X_M > t_0 | t_a > t_0; t_b > t_0)$.

- Step 2: the transgression risk estimation

The second step focuses on meeting a certain delay probability like in Goluch et al.³³ So, we note t_d the time at the latest is respected during an activity execution. This step objective is to estimate the probability $P(t_M > t_d)$, which specifies the delay transgression. It is possible to condition the activity duration in short by the elapsed time $P(t_M > t_d | t > t_0)$. This step is effective in estimating and controlling the business process since it anticipates and determines the timeout risk.

- Step 3: capturing the elapsed time

The last step, like in previous studies,^{22;64;65} our NM-WSPN model intended to be stochastically verified must have a monitoring system that captures and records the business process steps. On this assumption basis, the time elapsed since the previous event represents a information source that favors the preacher to obtain an accurate precision for the model. It is possible to illustrate the conditional effects on the remaining time with durations distribution. The formal representation is as follows:

- * $t \in TD$: time space,
- * $tr \in Tr$: timed transition,
- * $Distrib(tr)$: the duration distribution function,
- * $Density(tr)$: the density function is then the differential function $Distrib(tr)$.
- * $truncatcd_{tr}$: the truncated distribution function.

In a formal way, we denote that $t \in TD$ represents time and $tr \in Tr$ is a stochastic transition with the assigned duration distribution. Let $t_0 \in TD$; $t_0 > 0$ be tr the current transition time. Let $truncatcd_{\Delta}$ be the *truncate delta function*, which captures the entire probability mass at a single point.

We aim to build an algorithmic prediction solution based on the events immediate detection. It is assumed that the duration of each activity is purely isolated from other activities independent. This hypothesis makes it possible to simplify the forecasting process while remaining within the other prediction approaches norms.

To perform the forecast on an activity case, we build the forecast algorithm with six entries:

- * business process model, its *Markov Chain* representative or markings cardinalities graph,
- * current moment noted t_0 ,
- * maximum time limit t_d ,
- * minimum time limit t_m ,
- * Nbt : activity Cases total number,

- * activity current trace c that represents all the events observed up to the moment t_0 .

The prediction algorithm procedure is presented by the following organizational chart, explained in Figure 2.

The organizational chart describes the prediction process that starts with a case activity search operation in the model or its representative *Markov* chain. It is a graph *CardM* in the *NM-WSPN model* specifying all the observed events available in this case. We adopt the same idea of Ciardo et al.⁶⁶ and Anklesaria and Drezner³⁹; it is an alignment technique for localizing the current state by adjusting the transitions on the business process model. The next step is to collect the probabilistic simulation results (time at the latest, time at the earliest, the *CardMs* iterations number). During each iteration, the simulated case probabilistic distribution type is chosen based on a *NM-WSPN model* firing sequence.

It is assumed that all cases are independent, but the probability distribution of their representative transition is identical. The simulation operation is responsible for simulating trace sequences in the constrained sampling model based on conditioned truncated distributions without considering the transition origin *Distrib(tr)*. Then, it is necessary to raise again the conditioned truncated distributions *Truncated($t \mid t \geq t_0$)* samples.

The estimation operation is carried out with the empirical estimation method for the remaining durations (Figure 3).

5 | CASE STUDY (GELDZIN DEVELOPMENT WORKFLOW)

5.1 | Geldzin business process

We illustrate our method through *Geldzin DevOps* as a software development business process example.⁶⁷ This example is presented with a readable and comprehensible structure. Our modeling is restricted to the *DevOps* process steps (9, 10, 11, 12, 13, 14, 15, 16, and 17), which is realized by using *Bizagi Modeler*⁶⁸ as described in Figure 4. The following section illustrates the *NM-WSPN model* specification process.

5.2 | Subprocess specification by NM-WSPN model

We adopted a *BPMN* model manual transformation into an *NM-WSPN model*. This transformation uses the same rules presented in previous studies.^{21,69}

- The set $A = \{A_1, A_2, A_3, A_4, A_5\}$ represents the developers and the operators.

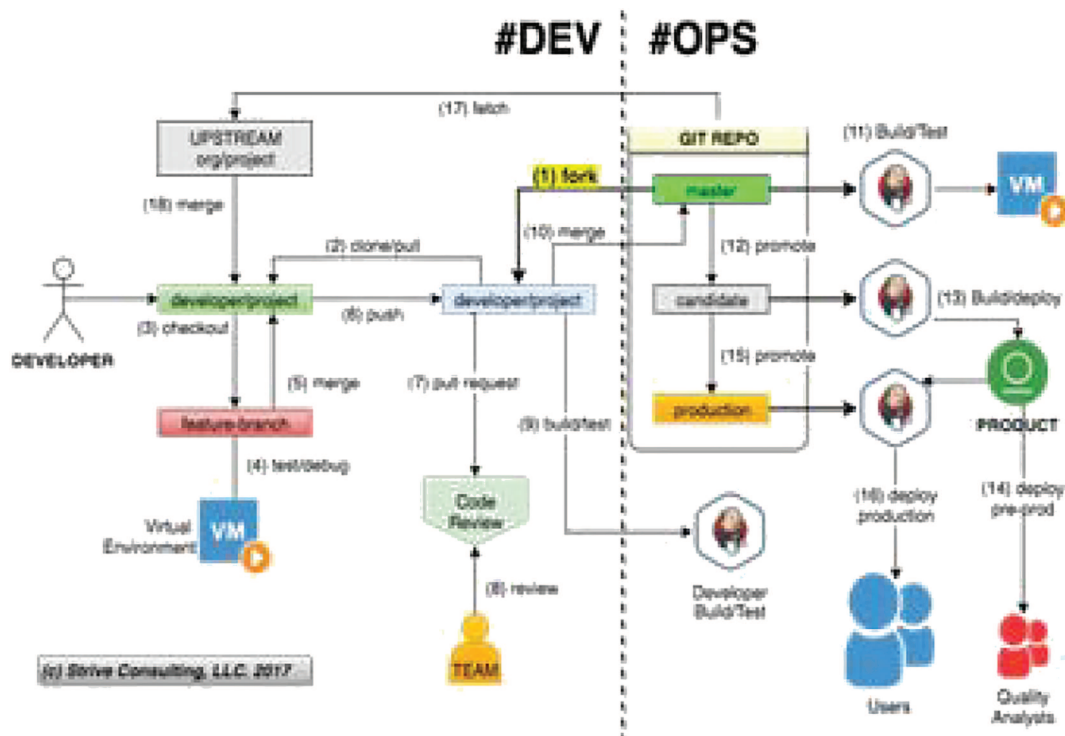


FIGURE 3 Geldzin development business process⁶⁷

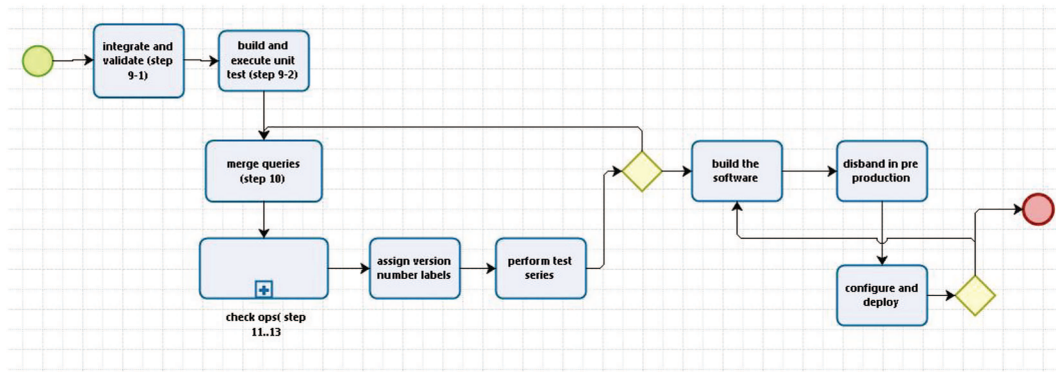


FIGURE 4 The business subprocess represented with Bizagi Modeler

- The specification AC represents the Geldzin development workflow software tool.
- The specification $SAC = \{Sac_1, Sac_2\}$ is made up of two subactivities resulting from the activity AC fragmentation.
- The *subprocess* model places specification:

The set $P = \{P_2, P_6, P_{13}, P_{14}, P_{15}, P_{16}, P_{17}, P_{18}, P_{19}, P_{20}, P_{21}, P_{22}, P_{23}\} \sqcup \{P_{24}, P_{25}, P_{26}, P_{27}, P_{28}, P_{29}, P_{30}, P_{31}, P_{32}\}$. The model places, without taking into account the intermediate places, are explained by Table 3:

The $\{P_{24}, P_{25}, P_{26}, P_{27}, P_{28}, P_{29}, P_{30}, P_{31}, P_{32}\}$ places set represents intermediate places when we adopt the splitting approach for nonimmediate transitions. Such places' role is to avoid the tokens loss during the transitions splitting as explained in Section 4.3. In the submodel specifying the Geldzin Workflow, place P_{12} is considered as an initial place.

- Transitions Specification:

According to the same approach in Ben Mesmia et al.,²⁹ the NM-WSPN model is liable to distinguish between three transition types (immediate, deterministic, and stochastic). Each stochastic or deterministic model transition is divided into three microtransitions (Tr^f , Tr^w , and Tr^p). the sub-model transition specification and the associated probability values meaning are well described in the Table 4.

According to Table 4, the transition Tr_{21} that represents Step 14, in the official website,⁶⁷ is described below ("The Jenkins job in step13 above also deploys the software into a pre-production environment where quality analysis and stakeholder preview can happen. Any feedback from this point that needs changes causes a jump back to step3.").

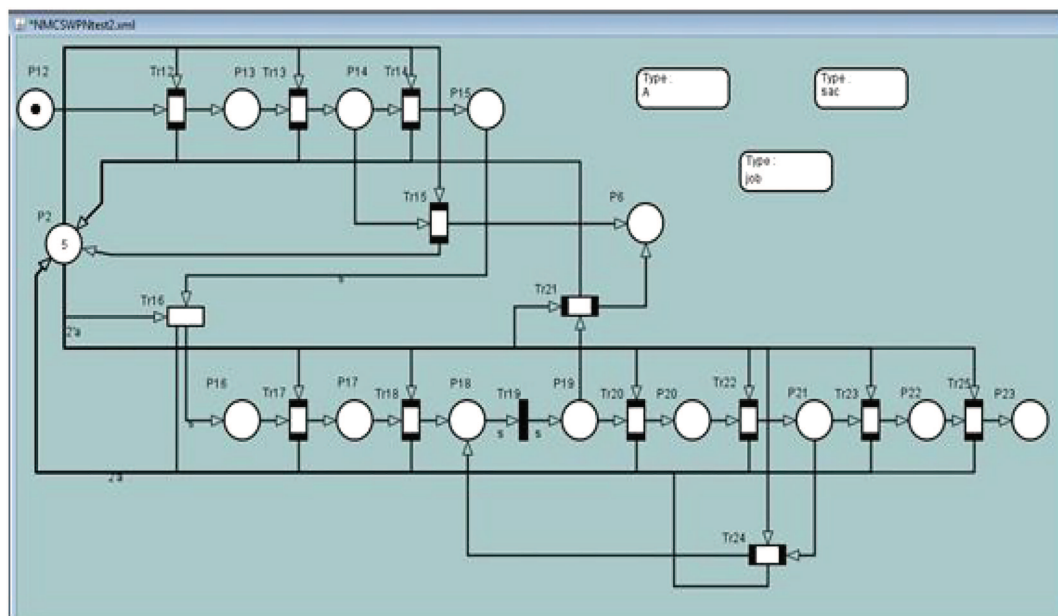
For example, during Tr_{21} firing, NM-WSPN model detects the transition type via the function calls $Distinct(Tr_{21}) = 3$ and $Typ - dist(Tr_{21}) = In$. We know that the overall firing time value is determined with the function call $Density^{-1}(Tr_{21}, In)$. The transition firing Tr_{21} , which is in the sub-activity executing charge Sac_1 , consists of sequentially firing the three microtransitions Tr_{21}^f , Tr_{21}^w , and Tr_{21}^p . The transition precondition T_{21}^c is the *uplet* $\langle 11, time, 1 \rangle$ result provided by the function call $Com(A_1, A_3, P_{19}, P_6, Tr_{21})$, the stochastic value $(0.33 * \varphi_{11})$, $MCom(A_1,$

TABLE 3 Places specification

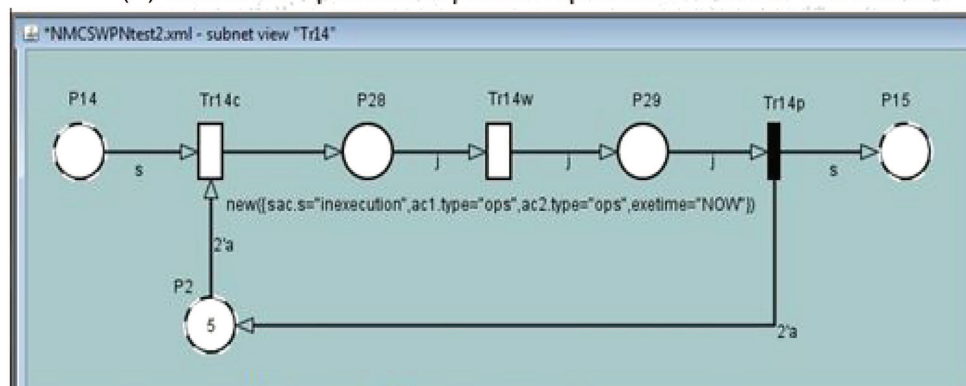
Places	Signification
P_2	All the actors developer-operators.
P_6	The source code after synchronizing.
P_{12}	The source code after the modifications validation.
P_{13}	The source code after the unit tests.
P_{14}	The source code after merging the queries.
P_{15}	Source code after software construction.
P_{16}	Source code after label assignment.
P_{17}	Source code without detecting undesirable characteristics.
P_{18}	The source code after the software publication.
P_{19}	Source code after deployment.
P_{20}	the source code after promotion in the "production" branch.
P_{21}	The source code after software deployment in production.
P_{22}	The source code after updating the environment with the current version requirements.
P_{23}	The source code after merging changes in "Master."

TABLE 4 The business subprocess transitions specification

Transition	Geldzin step	Transition type	Rate	Mets signification
Tr_{12}	9	Stochastic	φ_9	Construction success rate.
Tr_{13}	10	Stochastic	φ_{10}	Query merge success rate.
Tr_{14}	11	Stochastic	φ_{11}	Software testing success rate.
Tr_{15}	11	Stochastic	φ_{12}	Software test failure rate.
Tr_{16}	12	Deterministic		
Tr_{17}	12	Stochastic	φ_{13}	Labeling success rate.
Tr_{18}	12	Stochastic	φ_{14}	Undesirable detection rate.
Tr_{19}	13	Immediate		
Tr_{20}	14	Stochastic	φ_{15}	Deployment success rate.
Tr_{21}	14	Stochastic	φ_{16}	Deployment failure rate.
Tr_{22}	15	Stochastic	φ_{17}	Promising code success rate.
Tr_{23}	16	Stochastic	φ_{18}	Software deployment success rate.
Tr_{24}	16	Stochastic	φ_{19}	Software deployment failure rate.
Tr_{25}	17	Stochastic	φ_{20}	Development update success rate.



(A) Geldzin DevOps business process represented with NM-WSPN model

(B) Tr_{14} transition splitted into three micro-transitions**FIGURE 5** Geldzin DevOps business process NM-WSPN model and Tr_{14} transition split into three microtransitions

$Tr_{21}) = 1$, and $MCom(A_3, Tr_{21}) = 1$. The microtransition firing T_{21}^w is influenced by the stochastic value $(0.67 * \varphi_{11})$ and, formally, by the $uplet < 1, time, Sac_1 >$ resulting from the function $Execute(Sac_1, A_1, A_3, P_{19}, P_6, Tr_{21})$. The $MEx(A_1, Tr_{21}) = 1$ and $MEx(A_3, Tr_{21}) = 1$; the function call is done once T_{21}^w microtransition firing is achieved. The T_{21}^p firing is immediate; that is, the token Sac_1 is moving towards the place P_{16} , and the two tokens A_1 and A_3 will be released to the place P_2 (the original place before firing Tr_{21}).

5.3 | Geldzin business process model with NM-WSPN

In this subsection, we propose the NM-WSPN Petri nets model delineating the step specification already described in the previous subsection. The TimeNET tool⁷⁰ was used to present the Geldzin business process model in Figure 5.

From the DevOps workflow specified by our NM-WSPN model, we consider an abstraction with two levels:

The first level presents the graphical model representation via places, transitions, arc, tokens, and firing preconditions (see Figure 5A). The second level describes the micro places, the microtransitions, and their firing preconditions according to a microlevel life cycle (tokens consumption phase, execution phase, and tokens production phase). According to the split approach adopted by our NM-WSPN, each nonimmediate transition can be split into three microtransitions, and they can be examined according to the three possible distributions (Exponential, Normal, and Log-Normal) to detect, in the next, step the delays, and examine the execution performance criteria. To illustrate the microplaces and microtransitions, we present the abstraction second level applied to the transition Tr_{14} in Figure 5B. The model presented in Figure 3 shows the transition split mechanism that it adopts by our specification approach. The cardinality marking graph shown later is helpful to interpret and verify the properties following a transitions firing sequence.

5.4 | Geldzin business process formal verification

Our NM-WSPN model verification basis approach is the cardinality marking graph determination following the transitions firing simulation. This helps us to fix the model properties already defined in Section 4.3. Figure 6 illustrates the cardinality marking graph.

- The NM-WSPN model associated with the Geldzin business process for the firing sequence transition ($Tr_{12}..Tr_{25}$) is 5-bounded.
- The NM-WSPN model is not pure.
- The NM-WSPN model has no source transition.
- The transitions subset $\{Tr_{15}, Tr_{21}, Tr_{25}\}$ forms the well NM-WSPN model transitions.

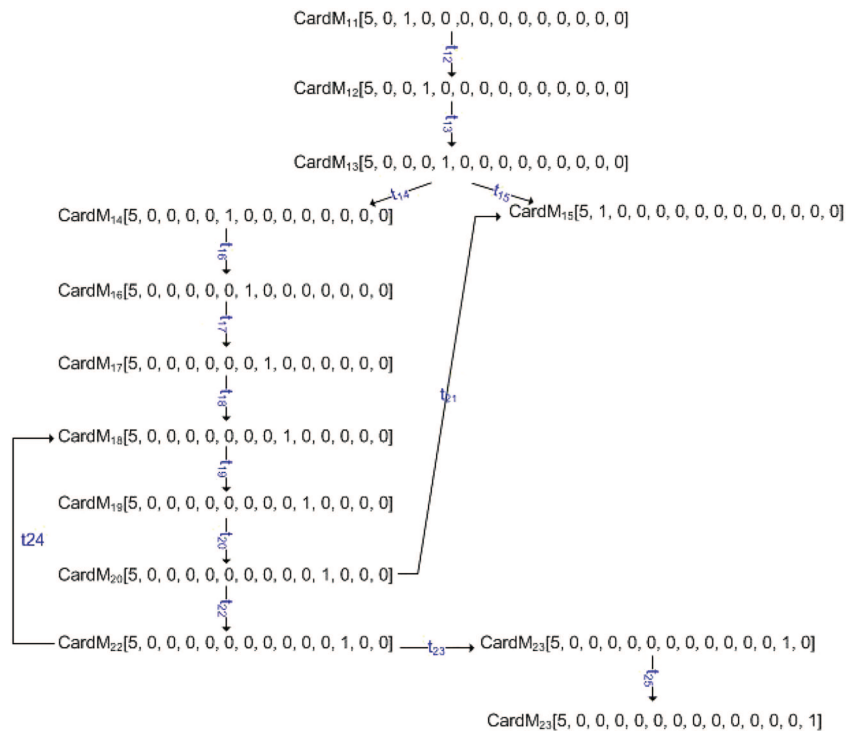


FIGURE 6 Geldzin business process NM-WSPN model cardinality marking graph

- All the transitions are *quasi-live* so the NM-WSPN model is almost alive.
- Note that the NM-WSPN model has an accessibility problem when it reaches $CardM_{15}$.
- We see that $CardM_{15}$ and $CardM_{21}$ are equivalent to what a firing circuit shows.
- The marking cardinality $CardM_{15}$ has no successor. So it is a blockage.
- The NM-WSPN model is *quasi-live*, not balanced and not sound.

Also, a qualitative verification is possible based on the two matrices $MCom$ and MEx that the NM-WSPN model provides after each transition firing. The sequence, allowing to succeed the sub activity execution in question, by the NM-WSPN submodel is $S_r = \{Tr_{12}, Tr_{13}, Tr_{14}, Tr_{16}, Tr_{17}, Tr_{18}, Tr_{19}, Tr_{20}, Tr_{22}, Tr_{23}, Tr_{25}\}$.

After the transition firing Tr_{25} , the model determines the two matrices transposition $C = MCom'$ and $E = MEx'$ as follows:

$$C_{25} = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \end{pmatrix} \quad E_{25} = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

Both matrices can help us rank the actors according to a communication and cooperation index. A_1 , A_2 , and A_4 actors participate in the DevOps process communication process 45÷. the actor A_5 is the most communicating among the actors set. Despite that, the actor A_3 is the least communicant, but he is most participating the execution with a rate equal to almost 72÷.

5.5 | Geldzin business subprocess simulation

A Cardinality Marking Graph interpretation verifies that $CardM_{15}$ is a blocking state or workflow process rejection. It is clear that two transition sequences $S_1 = \{Tr_{12}, Tr_{13}, Tr_{15}\}$ and $S_2 = \{Tr_{12}, \dots, Tr_{21}\}$ lead to the blocking state. In our firing analysis, we try to apply the method adopted by the NM-WSPN model. We select the transition sequence S_2 to determine the firing time for each transition in different scenarios that are dependent on the distributions probability type (Exponential, Normal, and Log-Normal). Our simulation solution must receive, as an input, a matrix specifying {transitions, times at the latest, times at the earliest, and firing rates} and the parameters related to the probability distribution type ($\lambda = 0.6$, $\mu = 0.6$, and $\sigma = 0.7$). Table 5 illustrates our simulation basic data.

We simulate each S_2 transition firing sequence by determining the real firing time using the $Density^{-1}(tr)$ function for the three possible distribution types, as shown in Table 6 and Figure 7.

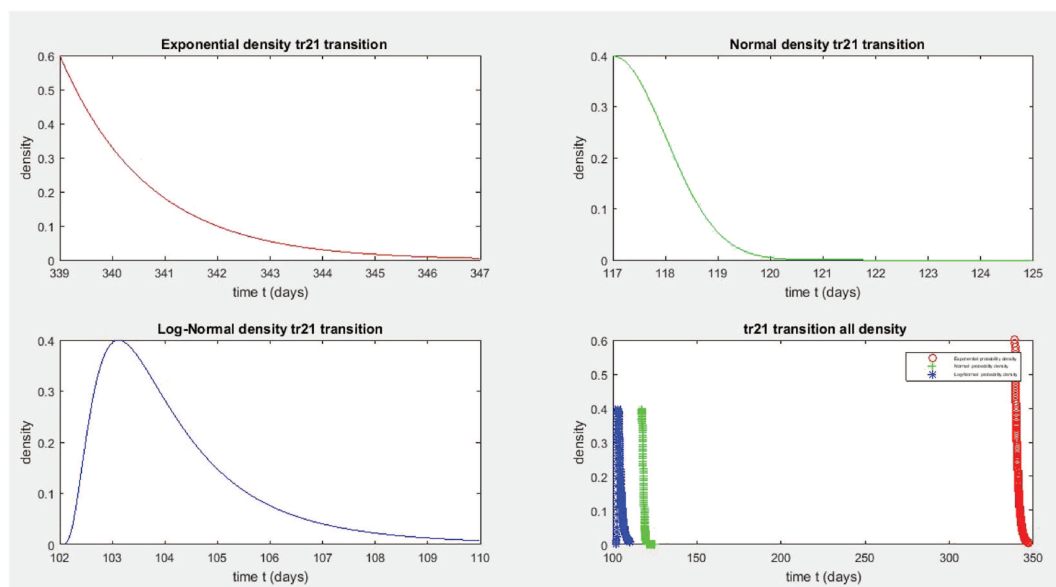
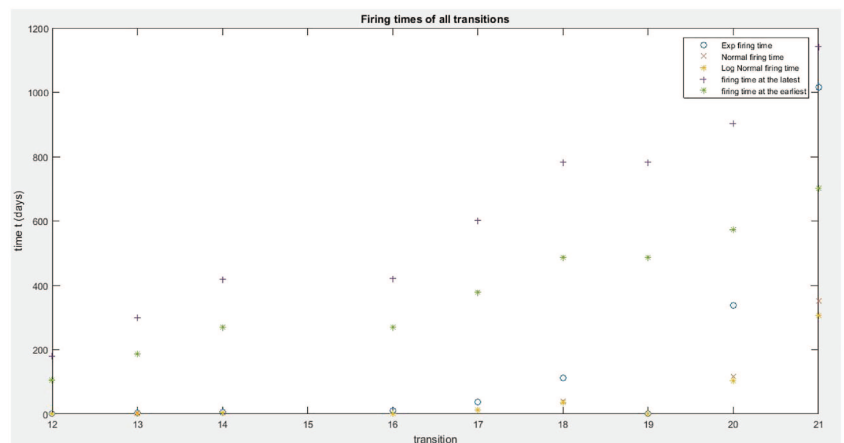
Our simulation approach solution is based on a transition vector composing the S_2 sequence. The program runs through the vector elements, alternating the density three types and calculating each transition three firing time. Algorithm 1 shows the firing time determination in the exponential density case.

TABLE 5 Simulation basic data

Transition	Firing rate	Time at the latest (month)	Time at the earliest (month)
Tr_{12}	0.3	6	3.5
Tr_{13}	0.3	4	2.75
Tr_{14}	0.7	4	2.75
Tr_{16}	0	0.07	0
Tr_{17}	0.3	6	3.65
Tr_{18}	0.3	6	3.6
Tr_{19}	0	0	0
Tr_{20}	0.3	4	2.85
Tr_{21}	0.3	8	4.3

TABLE 6 Each transition firing time matrix

Transition	EFT ⁽¹⁾	NFT ⁽²⁾	LNFT ⁽³⁾	CFTL (day) ⁽⁴⁾	CFTE (day) ⁽⁵⁾
Tr_{12}	34.2	22.77	19.8	180	105
Tr_{13}	102.6	68.31	59.4	300	187.5
Tr_{14}	136.8	91.08	79.2	420	270
Tr_{16}	273.6	2.1	2.1	422.1	272.1
Tr_{17}	1128.6	391.29	340.8	602.1	379.5
Tr_{18}	3385.8	1173.9	1022.4	782.1	487.5
Tr_{19}	0	0	0	782.1	487.5
Tr_{20}	10175	3521.6	3067.2	902.1	573
Tr_{21}	30472	10565	9201.6	1142.1	702

¹ EFT: Exponential firing time,² NFT: Normal firing time,³ LNFT: Log-Normal firing time,⁴ CFTL: Cumulative Firing time at the latest (day),⁵ CFTE: Cumulative Firing time at the earliest.**FIGURE 7** Firing times S_2 sequence transition**FIGURE 8** Tr_{21} transition different probability densities

The step results are stored in a matrix named *firingtimes*, which also contains the firing time at the earliest and the firing time at the latest. The comparison between the real firing time and the firing time at the latest makes it possible to locate the transition firing delays. Figure 8 shows the transition Tr_{21} densities.

Algorithm 1 ExpFiringtime Input: $tb, d, ph, \lambda \Rightarrow$ Output: eft

tb is the transition activation start time

d is the expected firing time

ph is the firing rate

Require : $ph \geq 0 \wedge d > 0$

$eft \leftarrow 0;$

if $ph == 0 \wedge d == 0$ **then**

$eft \leftarrow d$

else

$X \leftarrow x$

$N \leftarrow n$

if $d > 0$ **then**

$i \leftarrow tb$

while $i \leq td + d$ **do**

$y \leftarrow \lambda \times e^{-\lambda \times (i - tb)}$

if $y == ph$ **then**

$eft \leftarrow i$

Break

end if

$i \leftarrow i + 0.03$

end while

end if

$eft \leftarrow eft + tb$

end if

Return eft

5.6 | Geldzin business subprocess evaluation

Several evaluation axes are possible via our NM-WSPN model and the adjacent simulation. According to the properties formal verification results, it is clear that the NM-WSPN model is at 5-bounded, so the *Geldzin DevOps Workflow* process will not be enriched by other actors or by another subactivity. The NM-VPN model is not pure, which verifies that the process *DevOps* preserve their human resources (actors) all along these steps. The transitions subset $\{Tr_{15}, Tr_{21}, Tr_{25}\}$ firing forms the well NM-WSPN model transitions. Therefore, the $\{Tr_{15}, Tr_{21}\}$ changes the *en-execution* subactivity state to the state *suspended*. But, the transition firing Tr_{25} completes the *DevOps* process and updates the subactivity state to the *completed* state. The model has no source transition. That is, the *DevOps* process does not trigger in an autonomous and unpredictable way. Note that, the NM-WSPN model has an accessibility problem when it reaches $CardM_{15}$; in this case, the *DevOps* subactivities will be archived and will never be activated again. So it is a blockage. The transition $(Tr_{14}, Tr_{15}), (Tr_{20}, Tr_{21}), (Tr_{23}, Tr_{24})$ firing makes the model unsound and unbalanced but quasi-live.

Through the two matrices *MCom* and *MEx* analysis, we express the actors' performance and the cooperativity involved in the *DevOps* process.

The choice to simulate the sequence S_2 is based on the circuit presence. Moreover, it is clear that the first delay source is the transition Tr_{14} according to Table 5 and Figure 7. Our analysis and verification results are in line with the estimates shown in Figure 9.⁶⁷

In Figure 7, we present for each transition composing the S_2 sequence firing time at the earliest, the firing time at the latest, and the firing time estimation according to the three possible distribution adopted by the *NM-WSPN* model. In normal cases, the three firing time estimates must be framed by the earliest firing time and the latest firing time. The late transition is characterized by a firing time estimate, which exceeds the time limits at the latest.

In Figure 9, *Geldzin DevOps Workflow* designers examine their product according to three criteria: the trend test results, the convergence stages code, and convergence/complexity as a time function to completion.

5.7 | Geldzin business prediction delay

We present a *Geldzin DevOps Workflow* process explanatory summary of results through the use of the formal tools given to our *NM-WSPN* model, and a resource qualitative assessment put into play in the *Workflow*, by comparing them with the steps complexity and reliability estimates planned by the *DevOps Workflow* architects.

The model prediction approach requires the cardinality markings graph conversion to the *Event – log : Lev*, by matching each *CardM* in an event. According to the *Geldzin DevOps Workflow* simulation and verification results, the S_2 sequence firing causes a delay risk in the business process execution. It is proposed to make a prediction according to the algorithm approach illustrated in Figure 2. This operation purpose is to determine the firing transitions critical margin, which may be delayed. Our model performs a densities' truncation for the three possible distributions. This operation principle is illustrated by Algorithm 2, which holds only the truncated normal distribution.

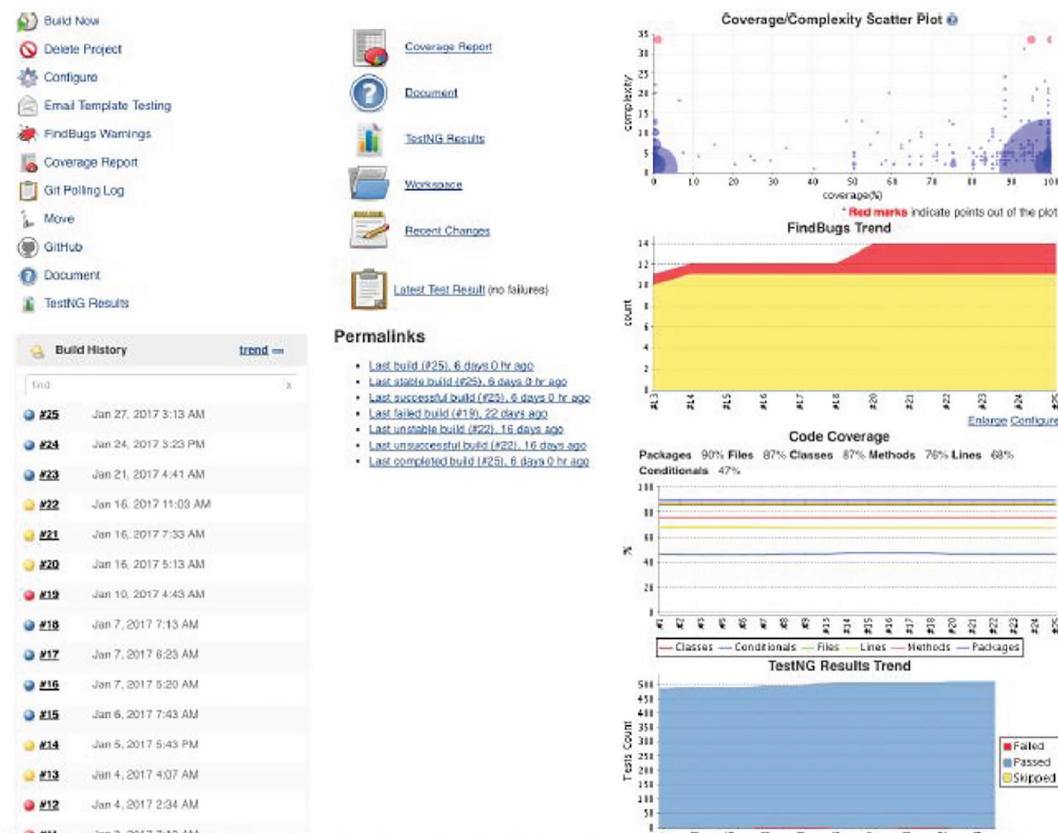


FIGURE 9 Geldzin DevOps Workflow performance analysis process⁶⁷

Algorithm 2 TruncateNormaldensity INPUT: $tc2, min2, max2 \Rightarrow$ Output: nt, ntt

$tc2$: truncation time

$min2$: truncations start time

$max2$: truncations finsh time

Normal density case

REQUIRE : $tc2 \geq min2 \wedge tc2 \leq max2$

.....

time vector initalization

.....

State $i \leftarrow 1$

while $i \leq size(t)$ **do**

$y \leftarrow \frac{1}{\sqrt{2 \times \pi}} \times e^{-0.5 \times |t-t1|^2}$

$z \leftarrow 1 - (0.5 \times (1 + erf(\frac{t1-t0}{\sqrt{2}})))$

$z1 \leftarrow 1 - (0.5 \times (1 + erf(\frac{t-t1}{\sqrt{2}})))$

erf : Gauss error function

calculates the truncated Normaldensity

if $t(i) < t1 \wedge z1 == 1$ **then**

$ntt(i) \leftarrow 0$

else

if $t(i) > t1 \wedge z1 < 1$ **then**

$ntt(i) \leftarrow \frac{y}{1-z}$

truncated Normaldensity

else

if $z == 1$ **then**

$tt(i) \leftarrow \frac{1}{\sqrt{2 \times \pi}} \times e^{-0.5 \times |t(i)-t1|^2}$

truncated _{Δ} Normaldensity

end if

end if

end if

$i \leftarrow i + 0.03$

end while

RETURN nt, ntt

An illustrative example of truncated densities is applied to Tr_{21} transition and is shown in Figure 10. According to our model transitions division approach, it is possible to apply the quietness at the level of microtransition, as it is illustrated in Figures 11 and 12.

The immediate microtransition Tr_{21}^p densities' truncation is not performed. The delay margin calculating operation is based on subtraction between the time corresponding to the already truncated density maximum and the truncation time. Then, a statistical population is prepared from observing the densities' truncation operations. For each density, the empirical mean and the empirical variance are calculated with the confidence interval constructing objective around the delay estimates with respect to the truncation time. We use the empirical estimation method, in Introduction to Inference Statistics,⁷¹ since the samples number is reduced.

Let n be the observations number, and $\{x_1, \dots, x_n\}$ is the observations set. The empirical mean is expressed by the following formula $\bar{x} = 1/n \sum x_i$. The empirical variance is obtained by the formula $\sigma^2 = 1/n - 1 \sum (x_i - \bar{x})^2$. An estimate trust interval is formally expressed by

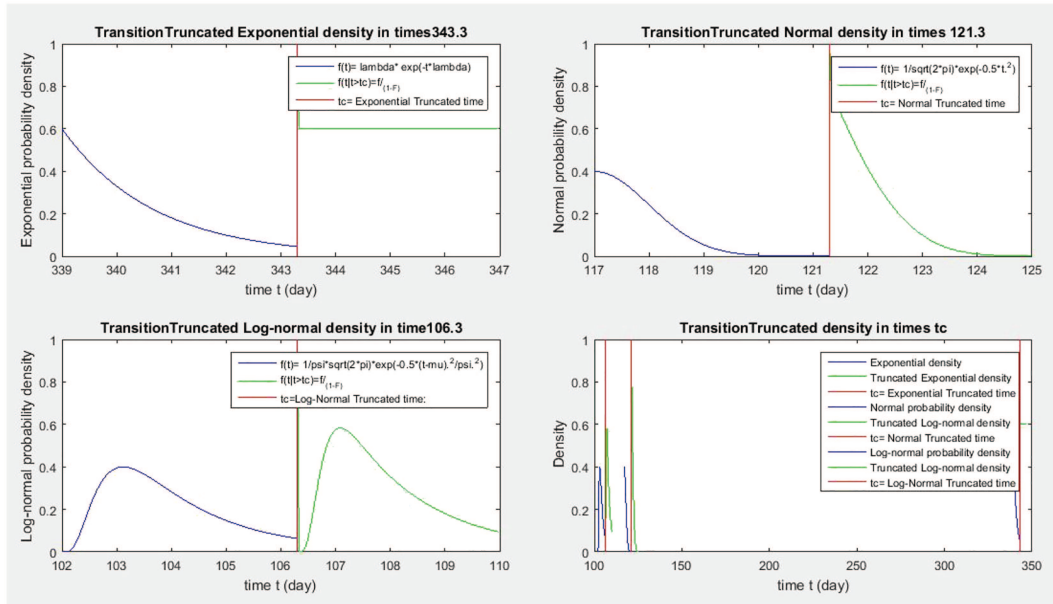


FIGURE 10 Tr_{21} all truncated densities

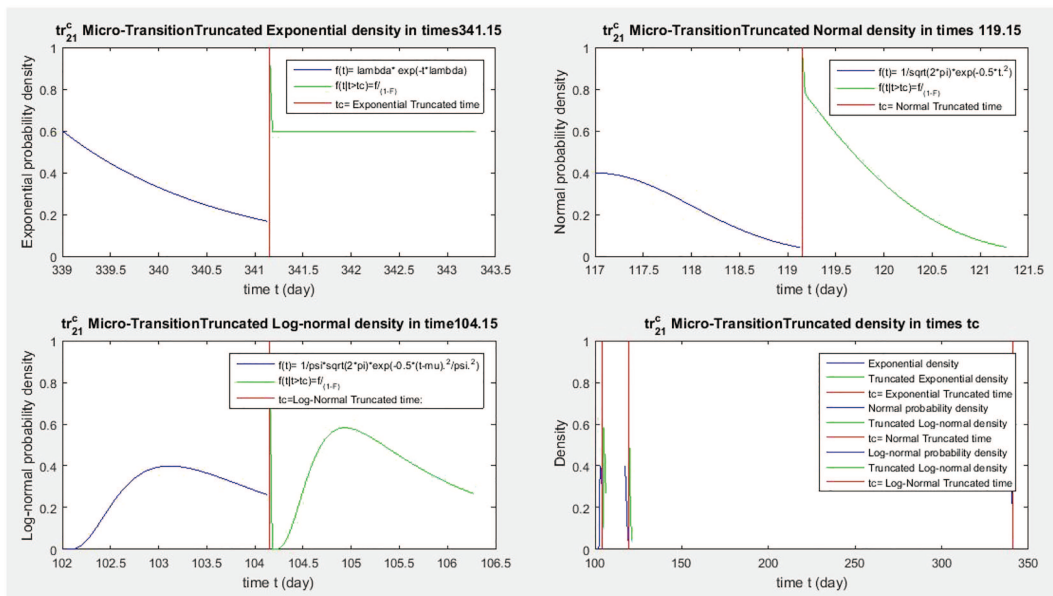


FIGURE 11 Tr_{21}^c microtransition all truncated densities

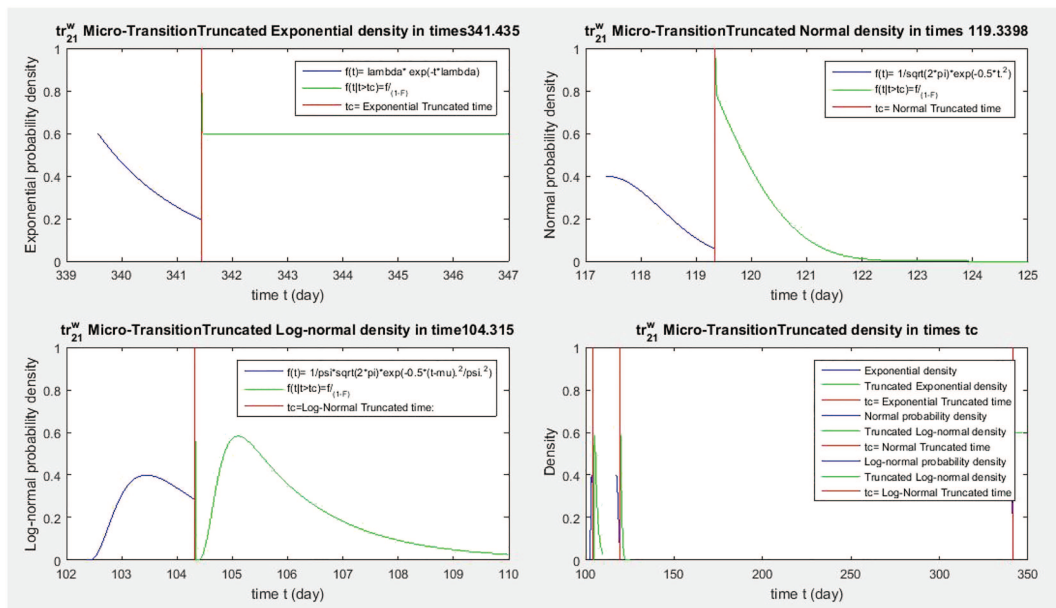


FIGURE 12 Tr_{21}^w microtransition all truncated densities

$$P(T - e \leq \Theta \leq T + e) = 1 - \alpha$$

T : Θ estimator,

e : error margin,

$1 - \alpha$: trust level,

α : error probability.

$\frac{\bar{x} - \mu}{\frac{\sigma}{\sqrt{n}}}$ converges to the normal laws $N(0,1)$.

Table 7 illustrates the prediction results based on our truncation method applied to Tr_{21} , Tr_{21}^c , and Tr_{21}^w micro transitions. Thus, the delay margins determined for all types of truncated density (Exponential, Normal, and Log-Normal).

TABLE 7 Prediction results by densities truncation

Transition	Density	TET ⁽¹⁾	TT ⁽²⁾	FT ⁽³⁾	DM ⁽⁴⁾	EM ⁽⁵⁾	EL ⁽⁶⁾	100*CM ⁽⁷⁾
Tr_{21}	Exp ⁽⁸⁾	347	343.3	343	±0.3	0.603	0.0013	±0.63
	N ⁽⁹⁾	125	121.3	121	±0.3	0.191	0.0551	±4.15
	L-N ⁽¹⁰⁾	110	106.3	106	±0.3	0.295	0.0321	±2.94
Tr_{21}^c	Exp	347	341.15	341	±0.15	0.6056	0.0022	±1.09
	N	125	119.15	119	±0.15	0.3216	0.0541	±5.41
	L-N	110	104.15	104	±0.15	0.39	0.0316	±4.13
Tr_{21}^w	Exp	347	341.43	341	±0.43	0.62	0.009	±0.42
	N	125	119.34	119	±0.34	0.125	0.044	±3.01
	L-N	110	104.315	104	±0.31	0.211	0.0345	±4.46

¹ TET: truncated end time,

² TT: truncated time,

³ FT: firing truncated maximum density,

⁴ DM: delay margin,

⁵ EM: empirical means,

⁶ EL: empirical likelihood,

⁷ CM: empiric confidence margin

⁸ Exp: exponential distribution,

⁹ N: normal distribution,

¹⁰ L-N: Log-normal distribution.

According to the predication results, based on the density truncation technique, adopted by our model, we note that if Tr_{21} is endowed with an exponential distribution, then the truncated density converges towards a uniform distribution, which explains the continuous delay from of ($TT = 343.4$) with a probability that exceeds 60% and a confidence empirical margin evaluated to ± 0.0063 . The delay probability is reduced in the normal distribution to 19.1% and log-normal 29.5%.

The empirical confidence interval⁷¹ determined by the *Central-Limit Theorem* is longer when the transitions have a normal distribution, except Tr_{21} ^w, which is the exception with a reduced prediction interval.

Similarly, our prediction method tools are based on maximum likelihood in the estimation phase and, then, they target a large number of transitions. Our empirical method use is justified by the transitions' reduced number and our firing semantics based on the *non-immediate* transitions division into three microtransitions.

6 | CONCLUSION

In this paper, we have developed a model named NM-WSPN as a sound NM-SPN⁴⁴ and *Workflow Nets*³⁶ combination. We show its importance in the specification, verification (soundness), and business processes software development performance evaluation, and we illustrate it on *Geldzin DevOps Workflow*. The formal transitions firing mechanism is not based only on the stochastic and deterministic aspect and on other probabilistic laws. The proposed model focuses on the dynamic interactions between the *Workflow* actors. Also, its semantics enables different detailed description steps involved in the *Workflow* execution. These steps are formally represented by microtransitions and concern, respectively, the tokens consumption, the task execution (subactivity), and the tokens production. Such a representation makes it possible to distinguish and locate, in a precise and fine manner, any execution failure kind.

The verification process adopted by the NM-WSPN model can be justified by a state space based on the markings cardinality graph to control and locate the shared resources conflicts.

The model is enriched with a formal tool, based on the elapsed time, to predict the remaining time and estimate the execution time during a transitions sequence. The assumptions about the elapsed time conditions are noticed when a long period of time is detected between the observable events in a business process. In our proposed model, the concurrent process execution is better described than the transition system, which is based on the method adopted in Van der Aalst et al.³⁵ The interorganizational process analysis shows that the process-related data have a significant impact on the activities' remaining time, and the decisions are taken.

In the framework of our future work, we intend, on the one hand, to enrich the NM-WSPN model expressiveness by integrating the resource constraints sharing⁷² and dependencies between the activities' time.⁷³ This can be done using the transformation techniques model, Fluid Stochastic Petri Net models (FSPN),^{61;69} or *RecatNets*¹⁵ models, which can lead to the tool automatically transforming the development of the BPMN model into an NM-WSPN model.²¹

On the other hand, to cover a dynamic *Workflow* context, the NM-WSPN model can be enriched by formal tools ensuring real-time performance dynamic display and *Workflow* dynamic criteria verification to effectively detect the system strengths and weaknesses, the interaction between the actors, and the customized content dissemination.

ORCID

Walid Ben Mesmia  <https://orcid.org/0000-0003-1270-5916>

REFERENCES

1. Humble J, Molesky J. Why enterprises must adopt devops to enable continuous delivery. *Cut IT J*. 2011;24(8):6.
2. Humble J, Farley D. *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Boston: Addison-Wesley Professional; 2010.
3. Oppenheimer D, Ganapathi A, Patterson DA. Why do internet services fail, and what can be done about it In: Proceedings of the 4th Conference on USENIX symposium on internet technologies and systems (USITS 2003), USITS 2003; 2003.
4. Delaet T, Joosen W, Vanbrabant B. A survey of system configuration tools. In: Proceedings of the 24th international conference on large installation system administration (LISA 2010), LISA; 2010.
5. Morris K. *Infrastructure as code: managing servers in the cloud*. Sebastopol: O'Reilly Media; 2016.
6. Herry H, Anderson P, Wickler G. Automated planning for configuration changes. In: Proceedings of the 25th international conference on large installation system administration (LISA 2011), USENIX 2011; 2011:57-68.
7. Hüttermann M. *DevOps for Developers*: Apress; 2012.
8. Wettlinger J, Breitenbucher U, Kopp U, Leymann F. Streamlining DevOps automation for Cloud applications using TOSCA as standardized metamodel. *Future Gener Comp Sys*. 2016;56(2016):317-332.
9. Zia B, Lapouchnian A, Yu E. Modeling of DevOps deployment choices using the design dimensions of process architecture. In: IFIP working conference on business modeling practice. Springer; 2015; Cham.
10. Nelson-Smith S. *Test-Driven Infrastructure with Chef*: O'Reilly Media Inc; 2013.

11. Adriansyah A, Van Dongen BF. Replaying history on process models for conformance checking and performance analysis. *WIREs: Data Min Know Disc.* 2012;2:182-192.
12. Weske M. *Business Process Management: Concepts, Languages, Architectures*: Springer; 2012.
13. La Rosa M, Mendling J, Reijers HA. *Fundamentals of Business Process Management*: Springer; 2013.
14. Mutschler B, Reichert M, Bumiller J. Unleashing the effectiveness of process-oriented information systems: problem analysis, critical success factors, and implications, Systems, Man, and Cybernetics. *PartC: Appl Reviews, IEEE Trans.* 2008;2008:280-291.
15. Barkaoui K, Hicheur A. Towards analysis of flexible and collaborative workflow using recursive ECATNets. In: BPM 2007 Workshops ter Hofstede A, Benattallah B, Paik H-Y, eds., LNCS 4928. Springer-Verlag; 2008:232-244.
16. Kheldoun A, Barkaoui K, Ioualalen M. Formal verification of complex business processes based on high-level Petri nets. *Inform Scie.* 2017;385:39-54.
17. BPMN. <http://www.bpmn.org/> [accessed: 24-01-2020].
18. BPMN 1.0.. <https://www.omg.org/spec/BPMN/1.0/About-BPMN/> [accessed: 24-01-2020].
19. El Kadiria S, Grabot B, Thobenc KD, Emmanouilidis C, Cieminskif G, Kiritsisa D. Current trends on ICT technologies for enterprise information systems. *Comp Indus.* 2016;79:14-33.
20. Barkaoui K, Ayed RB, Sbaili Z. Workflow soundness verification based on structure theory of Petri nets. *Int J Comput Inf Sci.* 2007;5(1):51-61.
21. Mutarraf U, Barkaoui K, Zhiwu L, Naiqi W, Ting Q. Transformation of business process model and notation models onto petri nets and their analysis. *Advan Mechan Eng.* 2018;10(12):1-21.
22. Andreas Rogge S, Mathias W. Prediction of business process durations using non-Markovian stochastic Petri nets. *J Inform Sys.* 2015;54:1-14.
23. WFMC. Workflow Management Application Programming Interface (interface 2 and 3) specification, 02. Wfmc-tc-1009; 1998.
24. Yamaguchi S, Yamaguchi M, Tanaka M. A soundness verification tool based on the SPIN model checker for acyclic workflow nets. ITC-CSCC; 2008.
25. Barkaoui K, Ben Ayed R. Uniform verification of workflow soundness. *Trans Instit Measur Cont.* 2011;33(1):133-148. <https://doi.org/10.1177/0142331208095676>
26. De Giacomo G, Vardi MY. Ltlf and Idlf synthesis under partial observability. *IJCAI.* 2016;2016:1044-1050.
27. Torres J, Baier JA. Polynomial-time reformulations of LTL temporally extended goals into final-state goals. *IJCAI.* 2015;2015:1696-1708.
28. Camacho A, Triantafillou E, Muise C, Baier JA, McIlraith S. Non-deterministic planning with temporally extended goals: LTL over finite and infinite traces. AAAI; 2017.
29. Ben Mesmia W, Escheikh M, Barkaoui K. DevOps Workow Specification based on Non-Markovian Stochastic Petri Nets with Enhanced Expressiveness. In: 6th COSI'2019; 2019; Tizi-Ouzou, Algrie:2-12. ISSN:978-9947-34-164-3.
30. Rosemann M, Muhlen MZ. Integrating risks in business process models. In: ACIS 2005 Proceedings; 2005.
31. Suriadi S, Weiss B, Winkelmann A, ter Hofstede AH, Adams M, Conforti R, Fidge C, Rosa ML, Ouyang C, Rosemann M, Pika A, Wynn M. Current research in risk-aware business process management: overview, comparison, and gap analysis. *Assoc Infor Sys.* 2014;34(1):933-984.
32. Pika A, Van der Aalst WMP, Fidge CJ, Ter-Hofstede AHM, Wynn MT. Predicting deadline transgressions using event logs. In: BPM Workshops 2012, V 132 of LNBIP. Springer; 2013:211-216.
33. Goluch G, Ekelhart A, Fenz S, Jakoubi S, Tjoa S, Muuck T. Integration of an ontological information security concept in risk aware business process management. In: HICSS-41; 2008:377-386.
34. Chauvet JM. Web Services avec SOAP, WSDL, UDDI, ebXML. Editions Eyrolles; 2002.
35. Van der Aalst WMP, Schonenberg MH, Song M. Time prediction based on process mining. *Inform Sys.* 2011;36(2):450-475.
36. Folino F, Guarascio M, Pontieri L. Discovering Context-Aware Models for Predicting Business Process Performances. In: OTM 2012. Springer; 2012: 287-304.
37. Hwang SY, Wang H, Tang J, Srivastava J. A Probabilistic Approach to Modeling and Estimating the QoS of Web-Services-Based Workows. *Infor Sys.* 2007;177(23):5484-5503.
38. Zheng H, Yang J, Zhao W, Bouguettaya A. QoS Analysis for Web Service Compositions Based on Probabilistic QoS. In: Service-Oriented Computing 2011. Springer; 2011:47-61.
39. Anklesaria KP, Drezner Z. A multivariate approach to estimating the completion time for PERT networks. *J Oper Res Soc;* 37(8):811-815.
40. Rozinat A, Wynn MT, Van der Aalst WMP, ter Hofstede AHM, Fidge CJ. Workow simulation for operational decision support. *J Data and Know Engin.* 2009;68(9):834-850.
41. Rozinat A, Mans RS, Van der Aalst WMP. Discovering simulation models. *J Inform Sys.* 2009;34(3):305-327.
42. Balbo G, Chiola G. Stochastic Petri Net Simulation. In: Proceedings of the 21st conference on Winter simulation. ACM; 1989:266-276.
43. Hammer M, Champy J. *Reengineering the Corporation: A Manifesto for Business Revolution*: Harper business Essentials; 2003.
44. Van der Aalst WMP, Van Hee K. *Workflow Management: Models, Methods, and System*: The MIL Press; 2002. ISSN: 0-262-01189-1.
45. Axenath B. The Aspects of Business Processes: An Open and Formalism Independent Ontology, Department of the University of Paderborn; 2005.
46. Akazi-Technologies. Business Process Management et Administration Electronique. In: Proceedings of the 4th Conference Technoforum: Akazi Technologies November; 2002. <https://www.akazi.com/>
47. Brathauget TA, Evjen TA. Enterprise Modeling. STF 38, isbn = A96302; 1996.
48. Jabbari R, bin Ali N, Petersen K, Tanveer B. What is DevOps? In: Scientific Workshop Proceedings on Agile Conference 2016. ACM Press; 2016; New York, USA.
49. Franča DB, Jeronimo H, Travassos GH. Characterizing DevOps by Hearing Multiple Voices. In: Proceedings of the 30th Brazilian Symposium on Software Engineering, 2016. ACM Press; 2016; New York, USA.
50. Peterson JL. *Petri net theory and the modeling of systems*: Prentice Hall. Englewood Clis; 2014.
51. AjmoneMarsan M, Balbo G, Conte G. A class of generalized stochastic Petri nets for the performance analysis of multiprocessor systems. *ACM Trans Comp Sys.* 1984;2:93-122. ISSN: A96302.
52. AjmoneMarsan M, Balbo G, Conte G, Donatelli S. *Modelling with Generalized Stochastic Petri Nets*: John Wiley and Sons; 1995.
53. Lindemann C. *Performance Modelling with Deterministic and Stochastic Petri Nets*: John Wiley and Sons; 1998.
54. Choi I, Song M, Park C, Park N. An XML-based process definition language for integrated process management. *Comp Indus.* 2003;50:85-102.
55. Choi H. Performance and reliability modelling using Markov regenerative stochastic Petri nets. *Phd thesis*: Graduate school of Duke University; 1993.

56. Ciardo G, Lindemann C. Analysis of deterministic and stochastic Petri nets. In: Proceedings of the 5th International Workshop on Petri Nets and Performance Models (PNPM'93). IEEE Comp Soc Press; 1993; france.
57. Dugan J, Trivedi K, Geist R, Nicola V. Extended Stochastic Petri Nets: Applications and Analysis. Performance 84, france, Proceeding Performance; 1984.
58. Ciardo G. Petri nets with marking-dependent arc cardinality: properties and analysis. In: Proceedings of the 15th International Conference on Application and Theory of Petri Nets 1994, Lectures Notes in Computer Science. Springer Verlag; 1994.
59. Ciardo G. Discrete-time markovian stochastic Petri nets. In: Proceedings of the 2th International Workshop on Numerical Solution of Markov Chains; 1995.
60. Bobbio A, Telek M. Non-exponential stochastic Petri nets: an overview of methods and techniques. *Com Sys Sci Eng*. 1997.
61. Trivedi K, Kulkarn V. i. FSPNs: fluid stochastic Petri nets. In: First 1 International Conference on Application and Theory of Petri Nets; 1993; Chicago: 20-31.
62. Cox DR. The analysis of non-Markovian stochastic processes by the inclusion of supplementary variables. *Proce Camb Philosopical Soc*. 1995;51: 433-440.
63. Rogge-Solti A, Van der Aalst WMP, Weske M. Discovering stochastic petri nets with arbitrary delay distributions from event logs. In: BPM Workshops, LNBIP, vol. 171. Springer; 2014; Heidelberg:15-27.
64. Van Dongen BF, Crooy RA, Van der Aalst WMP. Cycle Time Prediction: When Will This Case Finally Be Finished? In: OTM. Springer; 2008:319-336.
65. AjmoneMarsan M, Balbo G, Bobbio A, Chiola G, Conte G, Cumani A. The effect of execution policies on the semantics and analysis of stochastic Petri nets. *IEEE Trans Soft Engin*. 1989;se-15:832-846.
66. Ciardo G, German R, Lindemann C. A characterization of the stochastic process underlying a stochastic Petri net. *IEEE Trans Soft Engin*. 1994;se-20: 506-515.
67. Geldizin softwhere development. <https://blog.strive-ltd.com/category/software-development/devops/> [accessed: 2018-04-10].
68. Bizagi Modeler. <https://www.bizagi.com/en/products/bpm-suite/modeler/> [accessed: 2018-03-11].
69. Zimmermann A. Modeling and Evaluation of Stochastic Petri Nets with TimeNET 4.1. In: VALUETOOLS'12. IEEE; 2012:54-63.
70. TimeNet. <https://timenet.tu-ilmenau.de/> [accessed: 17-08-2019].
71. Introduction to Inference Statistics. https://www.lmd.polytechnique.fr/sturquet/teaching_data/mu001/chap3.pdf [accessed: 21-08-2019].
72. Barkaoui K, Petrucci L. Structural analysis of workflow nets with shared resources. In: Proceedings of workflow management: net-based concepts, models, techniques and tools (WFM98) van der Aalst WMP, De Michelis G, Ellis CA, eds., Computing science reports, vol. 98/7. Eindhoven University of Technology; 1998; Eindhoven:82-95.
73. Barkaoui K, Boucheneb H, Hicheur A. Modelling and Analysis of Time-Constrained Flexible Workflows with Time Recursive ECATNets. In: WS-FM 2008. LNCS Bruni R, Wolf K, eds., Vol. 5387. Springer; 2009; Heidelberg:19-36.

How to cite this article: Ben Mesmia W, Escheikh M, Barkaoui K. DevOps workflow verification and duration prediction using non-Markovian stochastic Petri nets. *J Softw Evol Proc*. 2020;e2329. <https://doi.org/10.1002/smr.2329>